FPT UNIVERSITY

FPT UNIVERSITY
HO CHI MINH CITY CAMPUS

FACULTY OF IC DESIGN AND DIGITAL AUTOMOTIVE ENGINEERING

PROJECT REPORT

DESIGN AND IMPLEMENTATION OF A MEDIAN–SOBEL IMAGE PROCESSING ACCELERATOR ON AN FPGA SoC PLATFORM

Supervisor: Phạm Thế Vinh

No.	Student Name	Student ID
1	Nguyễn Trí Tuệ	SE205019
2	Lê Nhất Nam	SE205267
3	Trần Lê Tiến Đạt	SE205121
4	Hà Gia Bảo	SE205134
5	Tống Trần Đăng	SE204445

Implementation Period: 08 September 2026 – 01 November 2026

Ho Chi Minh City, 2026

ACKNOWLEDGEMENTS

First and foremost, our team would like to express our sincere gratitude to the lecturers of FPT University, Ho Chi Minh City Campus, and especially to the Faculty of IC Design and Digital Automotive Engineering for providing us with the fundamental knowledge, academic environment, and technical foundation necessary to carry out this project.

In particular, we would like to express our deepest appreciation to our supervisor, **Phạm Thế Vinh**, for his dedicated guidance, valuable comments, and continuous support throughout the development of this project. His technical advice and practical suggestions during the design, implementation, verification, and debugging stages have helped our team gain a deeper understanding of FPGA-based digital system design, hardware–software integration, image-processing architectures, and the complete development flow of an SoC-based hardware accelerator.

We would also like to thank our classmates and friends who supported us through technical discussions, suggestions, and encouragement during the implementation and verification process.

Although our team has made every effort to complete the project carefully, our knowledge and practical experience are still limited. Therefore, the report and the implemented system may still contain shortcomings. We sincerely welcome constructive comments and suggestions from our lecturers so that we can continue improving our technical knowledge, engineering skills, and research experience in future projects.

Once again, our team would like to express our sincere gratitude to everyone who contributed to the completion of this project.

Ho Chi Minh City, 2026

The Project Team

ABSTRACT

In recent years, the growing demand for real-time image processing in embedded systems has increased the need for computing platforms that provide high performance, deterministic operation, and efficient hardware utilization. Common image-processing operations such as noise removal and edge detection require repeated neighborhood-based computations over large numbers of pixels, which may become inefficient when executed entirely by general-purpose processors. FPGA-based hardware acceleration therefore provides an effective solution by enabling customized data paths, pipelined processing, and direct integration with embedded processing systems.

This project focuses on the design and implementation of a Median–Sobel image-processing accelerator on an FPGA SoC platform. The complete processing flow begins with RGB-to-grayscale conversion and deterministic salt-and-pepper noise generation in the software preprocessing stage. The hardware processing pipeline then performs 3×3 Median filtering, 3×3 Sobel edge detection, gradient-magnitude computation using $|G_x| + |G_y|$, and binary thresholding. A Python implementation is used as the golden reference model, while the hardware architecture is implemented in synthesizable Verilog HDL.

The hardware design is developed in two main phases. In Phase 1, a streaming Image DSP core is constructed from Line Buffer, Window Generator, Median Filter, Sobel Filter, Gradient Magnitude, and Threshold modules. The processing stages maintain the required pixel-coordinate and valid-signal alignment throughout the pipeline and produce the final valid output region of $(W - 4) \times (H - 4)$ pixels. In Phase 2, the verified streaming core is integrated into a memory-mapped accelerator consisting of an Arbiter, finite-state machine controller, Input Memory, Datapath, and Output Memory. The accelerator is then packaged with an AXI4-Lite slave interface and integrated with the ARM Processing System of the AMD Kria KR260 platform.

During the development process, individual RTL modules and complete processing subsystems were independently simulated and verified using dedicated testbenches before FPGA deployment. The Phase 1 streaming core achieved bit-exact agreement with the Python golden model for multiple image resolutions, including 512×512 and 640×480. The Phase 2 accelerator and AXI4-Lite end-to-end simulations also completed successfully, including multi-frame execution, illegal-access handling, error-response behavior, and active-low reset recovery.

The accelerator was subsequently deployed on the AMD Kria KR260, where Ubuntu Linux

on the ARM Processing System controls the hardware through memory-mapped AXI4-Lite transactions. Post-route implementation meets the 100-MHz timing constraint with a worst setup slack of $+0.322$ ns, while the implemented design uses 3,560 CLB LUTs, 3,406 registers, two Block RAM tiles, and no DSP slices. A native ARM C implementation was also completed and produced bit-exact output in all 20 repeated benchmark runs with a mean kernel time of 4.817 ms.

Physical FPGA execution successfully demonstrates configuration, register access, frame transfer, accelerator control, status polling, and output readback. Nevertheless, the noisy-image FPGA result is not yet fully bit-exact: 5 of 20 runs pass, whereas 15 runs contain the same single mismatch at coordinate $(28, 40)$. A software timer also recorded a mean START-to-DONE interval of 23.008 μs; however, this value is smaller than the 40.96 μs lower bound required to feed 4,096 input pixels at 100 MHz and is therefore not used as a validated accelerator-latency or speedup result. Future work focuses on resolving the intermittent hardware mismatch, obtaining cycle-accurate FPGA latency using a hardware counter or ILA, and replacing per-pixel AXI4-Lite transfers with a higher-bandwidth data path such as AXI DMA or AXI4-Stream.

Contents

ACKNOWLEDGEMENTS **i**

ABSTRACT **ii**

1 PROJECT OVERVIEW **1**

1.1 Introduction 1

1.2 Project Objectives 3

1.3 Scope and Limitations 4

2 THEORETICAL BACKGROUND **7**

2.1 Digital Image Processing Overview 7

2.2 RGB-to-Grayscale Conversion 8

2.3 Salt-and-Pepper Noise 9

2.4 Median Filter 3×3 9

2.5 Sobel Operator 3×3 10

2.6 Gradient Magnitude 10

2.7 Edge Thresholding 11

2.8 Streaming Image-Processing Architecture 11

2.9 Line Buffer and Sliding Window 12

2.10 AXI4-Lite Interface 12

2.11 AMD Kria KR260 FPGA SoC Platform 13

2.12 Related FPGA Image-Processing Work 13

2.13 Chapter Summary 14

3 SYSTEM DESIGN AND IMPLEMENTATION **15**

3.1 System Architecture Overview 15

3.2 Phase 1 Streaming DSP Core 17

3.2.1 Line Buffer Architecture 18

3.2.2 3×3 Sliding-Window Generator 19

3.2.3 3×3 Median-Filter Architecture 20

3.2.4 3×3 Sobel Datapath 21

3.2.5 Gradient Magnitude and Threshold Datapath . . . 22
3.2.6 Registered Latency and Throughput Summary . . . 22
3.3 Phase 2 Structured Accelerator . . . 23
3.3.1 Memory Ownership and Frame Memories . . . 24
3.3.2 Phase 2 Datapath Controller . . . 25
3.3.3 Top-Level Control FSM . . . 26
3.3.4 Arbiter and Internal Request/Response Bus . . . 27
3.4 AXI4-Lite Interface Implementation . . . 27
3.5 Accelerator Register and Memory Map . . . 28
3.6 KR260 Vivado Integration . . . 29
3.6.1 Physical Address Assignment . . . 30
3.7 Linux-Side Accelerator Control . . . 31
3.8 Design Summary . . . 32

4 VERIFICATION AND EVALUATION 33
4.1 Verification Methodology . . . 33
4.2 Module-Level Verification . . . 34
4.3 Phase 1 End-to-End Verification . . . 35
4.4 Phase 2 and AXI4-Lite Verification . . . 36
4.5 Post-Implementation Evaluation . . . 36
4.5.1 Routing and Resource Utilization . . . 36
4.5.2 Timing and Critical Path . . . 37
4.5.3 Power Estimation . . . 38
4.6 KR260 Hardware Evaluation . . . 38
4.6.1 MMIO Bring-Up and Black-Frame Test . . . 38
4.6.2 Golden-Model Consistency Check . . . 39
4.6.3 Native ARM C Baseline . . . 39
4.6.4 FPGA Timing Consistency and Repeated Benchmark . . . 40
4.6.5 Analysis of the Intermittent FPGA Mismatch . . . 41
4.7 Comparison with Related FPGA Work . . . 42
4.8 Verification and Evaluation Summary . . . 43

5 CONCLUSION AND FUTURE WORK 44
5.1 Achievements . . . 44
5.2 System Limitations . . . 45
5.3 Future Work . . . 46
5.4 Final Remarks . . . 47

REFERENCES 48

List of Figures

3.1 System-level architecture of the image-processing accelerator on the KR260 platform. 16
3.2 Architecture of the Phase 1 streaming image-processing pipeline. 17
3.3 Pixel datapath architecture of the line-buffer module. 18
3.4 Architecture of the 3×3 sliding-window generator. 19
3.5 Architecture of the 3×3 Median filter using a nine-stage combinational compare-swap network. 20
3.6 Datapath architecture of the 3×3 Sobel operator. 21
3.7 Datapath architecture of the gradient-magnitude and binary-threshold stages. . 22
3.8 Internal architecture of the Phase 2 image-processing accelerator. 24
3.9 Finite-state machine of the Phase 2 accelerator controller. 26
3.10 Vivado block design used to integrate the custom accelerator with the KR260 Processing System. 30

4.1 Numerical validation of the web-generated Python golden output against the original `expected.hex`; all 4,096 entries match. 39
4.2 Twenty-run KR260 benchmark log. ARM C and FPGA correctness results are used directly; the displayed FPGA timing and derived speedup are retained only as diagnostic software-timer output because they fail the cycle-count consistency check. 41

List of Tables

2.1 Execution location of each image-processing stage. 7
2.2 Valid regions of the two consecutive 3×3 processing stages. 8
2.3 Input and final valid output pixel counts. 8
2.4 AXI4-Lite communication channels. 13
2.5 Selected related FPGA image-processing implementations. 14

3.1 Principal design parameters. 17
3.2 Registered latency of the Phase 1 streaming datapath. 23
3.3 Responsibilities of the Phase 2 modules. 24
3.4 Memory ownership by accelerator state. 25
3.5 Outputs generated by the Phase 2 control FSM. 27
3.6 Logical address map of the image-processing accelerator. 28
3.7 CTRL register commands. 29
3.8 STATUS register bit definitions. 29
3.9 Physical KR260 addresses of the accelerator interface. 30
3.10 Software and data artifacts used in the verification and benchmark flow. 32

4.1 Verification levels used in the project. 34
4.2 Module-level RTL verification summary. 35
4.3 Phase 1 end-to-end simulation results. 35
4.4 Phase 2 simulation regression results. 36
4.5 Post-placement resource utilization of the implemented design. 37
4.6 Post-route timing results at 100 MHz. 37
4.7 Critical setup path through the Median-filter network. 38
4.8 Native ARM C kernel timing over 20 executions. 39
4.9 Overall verification and evaluation summary. 43

Chapter 1

PROJECT OVERVIEW

1.1 Introduction

In recent years, the rapid development of embedded systems, computer vision, autonomous systems, and intelligent edge devices has significantly increased the demand for real-time image-processing capabilities. Applications such as object detection, visual monitoring, industrial inspection, autonomous vehicles, and Internet of Things (IoT) devices often require image data to be processed directly at the edge instead of being transmitted to remote servers or cloud platforms. As a result, embedded computing systems are increasingly expected to provide not only sufficient processing performance but also deterministic operation, low latency, flexible hardware integration, and efficient use of computational resources.

Image filtering and edge detection are among the fundamental operations used in many computer-vision systems. Raw image data may contain impulse noise caused by transmission errors, image sensors, or external interference. Salt-and-pepper noise, in particular, introduces isolated pixels with extremely high or low intensity values and can significantly affect subsequent feature-extraction operations. Median filtering is commonly applied to remove this type of noise because it can suppress isolated outliers while preserving important image structures more effectively than simple linear averaging methods [1, 2].

After noise reduction, edge detection is often performed to identify regions in which image intensity changes significantly. Among classical edge-detection algorithms, the Sobel operator is widely used due to its relatively simple computational structure and its ability to estimate horizontal and vertical image gradients using local 3×3 neighborhoods [1, 7]. The resulting gradient information can be used to determine the locations of object boundaries and other important structural features in an image.

Although Median and Sobel filters can be implemented in software on general-purpose processors, both algorithms require repetitive neighborhood-based operations for every image pixel. For larger image resolutions or real-time applications, repeatedly accessing neighboring pixels and executing the same arithmetic operations may become a computational bottleneck. In con-

trast, Field-Programmable Gate Arrays (FPGAs) allow designers to construct dedicated data paths, exploit fine-grained parallelism, and organize image-processing operations as streaming pipelines. Once the pipeline has been filled, consecutive pixels can be processed continuously, making FPGA technology suitable for deterministic and high-throughput image-processing applications [7, 11].

Furthermore, modern System-on-Chip (SoC) FPGA platforms integrate a general-purpose processor and programmable logic within the same device. The processor can perform software-level control, memory management, communication, and system configuration, while the programmable logic executes computationally intensive operations using specialized hardware architectures. This hardware–software co-design approach combines the flexibility of software with the parallel processing capability of FPGA logic and provides a suitable platform for developing embedded hardware accelerators. The KR260 used in this project is based on the Kria K26 SOM and a Zynq UltraScale+ MPSoC, which integrates an Arm-based Processing System with programmable logic [5].

Based on these considerations, this project, entitled *"Design and Implementation of a Median–Sobel Image Processing Accelerator on an FPGA SoC Platform"*, focuses on developing a complete image-processing system that combines a software reference model, a streaming RTL processing core, a memory-mapped hardware accelerator, and an ARM-based control environment.

The image-processing algorithm consists of grayscale conversion, deterministic salt-and-pepper noise generation, 3×3 Median filtering, 3×3 Sobel edge detection, gradient-magnitude computation, and binary thresholding. A Python implementation is first developed as the golden reference model to define the expected processing behavior. The main processing stages are subsequently implemented in synthesizable Verilog HDL as a streaming hardware pipeline.

The verified streaming core is then integrated into a memory-mapped accelerator containing an Arbiter, finite-state machine controller, Input Memory, Datapath, and Output Memory. The accelerator is connected to the ARM Processing System through an AXI4-Lite interface [4] and deployed on the AMD Kria KR260 platform [5]. Ubuntu Linux running on the ARM Processing System is used to configure and control the accelerator through memory-mapped transactions.

Through this project, the team aims to strengthen its knowledge of digital image processing, RTL design, streaming hardware architectures, FPGA verification, AXI-based hardware–software communication, and SoC system integration. The implementation also provides a practical foundation for future studies involving more advanced image-processing accelerators, high-bandwidth data-transfer architectures, and CPU–FPGA performance comparisons.

Prior FPGA studies demonstrate several complementary design choices relevant to this project. Median-filter architectures emphasize the area–latency trade-off of order-statistic hardware [6, 10]; multiplier-free Sobel implementations show that shift-add gradient datapaths can avoid dedicated multipliers [7]; and recent Zynq-based systems illustrate both streaming image-processing integration and the importance of memory-interface architecture [12, 9, 13]. These works provide context rather than a direct ranking, because they use different algorithms, devices, frequen-

cies, interfaces, and verification conditions. A focused comparison is presented in Chapter 2 and revisited against the measured project results in Chapter 4.

1.2 Project Objectives

The project focuses on the design and implementation of an FPGA-based accelerator for classical image processing and its integration into an ARM–FPGA SoC system. The complete system is developed progressively from a software reference model to a verified RTL processing core and finally to a memory-mapped hardware accelerator deployed on the AMD Kria KR260 platform.

The main objectives of the project are as follows:

- Develop a deterministic Python golden model for the complete image-processing algorithm, including RGB-to-grayscale conversion, salt-and-pepper noise generation, Median filtering, Sobel edge detection, gradient-magnitude calculation, and binary thresholding.
- Design a synthesizable streaming Image DSP core in Verilog HDL consisting of the following functional blocks:
  - Line Buffer;
  - 3×3 Window Generator;
  - 3×3 Median Filter;
  - 3×3 Sobel Filter;
  - Gradient Magnitude;
  - Threshold.
- Design the processing pipeline so that pixel data, valid signals, and image coordinates remain correctly synchronized throughout all processing stages.
- Verify the streaming Image DSP core against the Python golden model for multiple image resolutions and ensure bit-exact agreement in RTL simulation.
- Develop a memory-mapped accelerator architecture around the verified streaming core. The accelerator includes:
  - an Arbiter for address decoding and host-access control;
  - a finite-state machine controller for accelerator operation;
  - Input Memory for storing image data;
  - a Datapath for supplying pixels to the streaming processing core;
  - Output Memory for storing the final valid edge pixels.

- Define a deterministic accelerator control sequence consisting of the IDLE, LOAD, EXEC, READ, and ERROR states, together with command handling, access permissions, completion signaling, and error recovery.
- Package the accelerator as an AXI4-Lite peripheral and establish communication between the Processing System (PS) and Programmable Logic (PL).
- Integrate the accelerator into a Vivado Block Design for the AMD Kria KR260 platform using the Zynq UltraScale+ MPSoC Processing System, AXI Interconnect, reset subsystem, and custom Image DSP IP.
- Develop an ARM Linux userspace control program to access the accelerator through physical memory mapping, transfer input pixels, issue control commands, monitor accelerator status, retrieve output pixels, and compare the FPGA result with the software reference.
- Perform functional verification at multiple abstraction levels, including individual RTL modules, the complete streaming pipeline, the Phase 2 accelerator, the AXI4-Lite interface, and the physical FPGA implementation.
- Evaluate the implemented system in terms of functional correctness, post-route timing, hardware-resource utilization, estimated power, and physical-board behavior.
- Develop and validate a native ARM C implementation of the same Median–Sobel algorithm as a processor-side performance and correctness baseline.

Through these objectives, the project aims not only to implement the Median–Sobel image-processing algorithm in hardware but also to establish a complete FPGA development flow covering algorithm modeling, RTL design, simulation, IP packaging, SoC integration, Linux-based software control, and physical hardware verification.

1.3 Scope and Limitations

To ensure that the project remains feasible and focused on the fundamental aspects of FPGA-based image-processing accelerator design, the implementation is limited to the following scope:

- The image-processing algorithm is limited to grayscale image processing, deterministic salt-and-pepper noise generation, 3×3 Median filtering, 3×3 Sobel edge detection, gradient-magnitude computation using

  $$M = |G_x| + |G_y|,$$

  and binary thresholding.

- RGB-to-grayscale conversion and salt-and-pepper noise generation are currently performed offline in the Python preprocessing stage. These operations are not implemented inside the FPGA accelerator.
- Pixel intensity is represented using 8-bit unsigned integer data. The final thresholded output is a binary image in which each valid output pixel is either `0x00` or `0xFF`.
- The threshold value used in the current implementation is fixed at

$$\texttt{THRESHOLD} = 100.$$

- The Median and Sobel filters both operate on 3×3 neighborhoods. As a result of the two consecutive neighborhood-processing stages, the final valid region excludes two border pixels from each side of the original image. For an input image of size $W \times H$, the number of valid output pixels is

$$N_{\text{output}} = (W - 4)(H - 4).$$

- The Phase 1 hardware implementation follows a streaming architecture. The algorithm, valid-region convention, coordinate mapping, and output order of this core are treated as fixed once bit-exact agreement with the Python golden model has been achieved.
- The Phase 2 accelerator uses on-chip memories to store input and output pixels. Therefore, the maximum hardware image size depends on the available FPGA BRAM resources and on the configured memory-address width.
- The current KR260 hardware bring-up configuration uses a 64×64 image with 4096 input pixels and 3600 valid output pixels. Larger resolutions have been verified in simulation, but the current physical accelerator configuration is intentionally kept small during hardware debugging.
- Pixel transfers between the ARM Processing System and the accelerator are currently performed through AXI4-Lite memory-mapped transactions. One 8-bit pixel occupies one 32-bit address slot with a 4-byte address stride.
- AXI DMA and AXI4-Stream are not included in the current implementation. Consequently, the present architecture is designed primarily for functional verification and hardware–software integration rather than maximum data-transfer bandwidth.
- Two separate native programs are used on the KR260: an MMIO control program for the FPGA accelerator and an optimized ARM C implementation of the Median–Sobel algorithm. The latter provides an independent processor-side reference and achieved bit-exact agreement in 20 of 20 repeated runs.

- Functional verification in RTL simulation has been completed for the principal Phase 1 and Phase 2 components. However, simulation correctness is treated separately from physical FPGA correctness.
- Communication between the ARM Processing System and the accelerator on the KR260 has been successfully established, including register access, input transfer, control-command execution, status monitoring, and output readback. For the noisy 64×64 input, the FPGA is bit-exact in 5 of 20 repeated runs; the other 15 runs contain one identical mismatch at coordinate $(28, 40)$. Therefore, the physical FPGA implementation remains under verification rather than fully validated.
- Post-route timing, resource utilization, and Vivado power estimation are reported. The ARM C kernel timing is also measured. A software-visible FPGA START-to-DONE value was recorded, but it is not accepted as a validated accelerator latency because it is smaller than the theoretical input-feed lower bound at 100 MHz. Consequently, no final CPU–FPGA speedup is claimed.
- Advanced image-processing functions, adaptive thresholding, color-image processing inside the FPGA, external-memory image buffering, DMA-based transfer, and complete computer-vision applications are outside the current scope and are reserved for future development.

Within these limitations, the project focuses on establishing a correct and verifiable hardware architecture for Median–Sobel image processing, together with a complete software-to-hardware control path on an FPGA SoC platform. The resulting system serves as a foundation for subsequent optimization, larger image resolutions, higher-bandwidth communication interfaces, and more comprehensive CPU–FPGA performance evaluation.

Chapter 2

THEORETICAL BACKGROUND

This chapter summarizes the theoretical concepts required for the proposed Median–Sobel accelerator. It covers the image-processing operations, streaming neighborhood processing, AXI4-Lite communication, the AMD Kria KR260 platform, and selected related FPGA implementations. Detailed RTL architecture and implementation are presented in Chapter 3, while verification and hardware results are discussed in Chapter 4.

Table 2.1 summarizes where each processing stage is executed.

Table 2.1: Execution location of each image-processing stage.

Stage	**Execution location**
RGB-to-grayscale conversion	Offline preprocessing in Python
Salt-and-pepper noise generation	Offline preprocessing in Python using a fixed random seed
Median filter 3×3	Python golden model and Verilog RTL
Sobel operator 3×3	Python golden model and Verilog RTL
Gradient magnitude	Python golden model and Verilog RTL
Binary threshold	Python golden model and Verilog RTL

2.1 Digital Image Processing Overview

A grayscale digital image can be represented as a two-dimensional array of discrete intensity samples [1]. In this project, each hardware pixel is an unsigned 8-bit value,

$$0 \leq I(x, y) \leq 255,$$

with the origin at the upper-left corner. Pixels are processed in raster order so that the software model, RTL simulation, frame memories, and physical FPGA execution use the same coordinate convention.

Both the Median and Sobel operators use a 3×3 neighborhood,

$$\begin{bmatrix} p_{00} & p_{01} & p_{02} \\ p_{10} & p_{11} & p_{12} \\ p_{20} & p_{21} & p_{22} \end{bmatrix}.$$

No padding is used. Therefore, the first 3×3 stage produces valid centers only for

$$1 \le x \le W - 2, \qquad 1 \le y \le H - 2,$$

while the second cascaded 3×3 stage contracts the final valid region to

$$2 \le x \le W - 3, \qquad 2 \le y \le H - 3.$$

Thus,

$$N_{\text{median}} = (W - 2)(H - 2), \qquad N_{\text{output}} = (W - 4)(H - 4).$$

Table 2.2 summarizes the two valid regions.

Table 2.2: Valid regions of the two consecutive 3×3 processing stages.

Stage	**Valid columns** x	**Valid rows** y	**Valid pixels**
Median 3×3	$1 \dots W - 2$	$1 \dots H - 2$	$(W - 2)(H - 2)$
Sobel 3×3	$2 \dots W - 3$	$2 \dots H - 3$	$(W - 4)(H - 4)$

For the resolutions used in verification, the corresponding frame sizes are shown in Table 2.3.

Table 2.3: Input and final valid output pixel counts.

Resolution	**Input pixels**	**Final valid pixels**
64×64	4,096	3,600
512×512	262,144	258,064
640×480	307,200	302,736

2.2 RGB-to-Grayscale Conversion

The input image is converted from RGB to grayscale before being sent to the accelerator. A luminance approximation consistent with the weighting used in ITU-R BT.601 is [3]

$$Y = 0.299R + 0.587G + 0.114B.$$

To obtain deterministic integer arithmetic, the coefficients are scaled by 256 and approximated as 77, 150, and 29. The implementation therefore uses

$$Y = \left\lfloor \frac{77R + 150G + 29B}{256} \right\rfloor = (77R + 150G + 29B) \gg 8. \tag{2.1}$$

Since $77 + 150 + 29 = 256$, the output remains in the 8-bit range. RGB-to-grayscale conversion is performed offline in Python, so the FPGA receives an already converted grayscale frame.

2.3 Salt-and-Pepper Noise

Salt-and-pepper noise is an impulse-noise model in which selected pixels are replaced by minimum or maximum intensity values [1]. The project uses a total corruption probability of 5%. For a uniformly distributed random value $r \in [0, 1)$,

$$I_N = \begin{cases} 0, & r < 0.025, \\ 255, & 0.025 \le r < 0.05, \\ I, & r \ge 0.05. \end{cases} \tag{2.2}$$

A fixed random seed of 12345 is used so that the noisy input is identical across the Python model, RTL simulation, ARM C implementation, and FPGA experiments. Noise generation is performed offline rather than inside the accelerator.

2.4 Median Filter 3×3

The Median filter is a nonlinear order-statistic filter widely used to suppress impulse noise while preserving significant edges [1, 2]. For each valid 3×3 neighborhood, the nine samples are sorted,

$$v_{(1)} \le v_{(2)} \le \cdots \le v_{(9)},$$

and the output is the fifth ordered value,

$$I_M(x, y) = \text{median}\{p_{00}, p_{01}, \ldots, p_{22}\} = v_{(5)}. \tag{2.3}$$

Because the filter selects an existing sample rather than averaging neighboring values, isolated salt-and-pepper impulses are often removed without smoothing edges as strongly as a linear averaging filter.

In hardware, the fixed nine-input ordering problem can be implemented using compare-and-swap networks. FPGA Median-filter studies show a trade-off between deeply pipelined parallel

networks and smaller iterative architectures [6, 10]. The specific combinational sorting network used in this project is described in Chapter 3.

2.5 Sobel Operator 3×3

The Sobel operator estimates horizontal and vertical intensity gradients using two 3×3 kernels [1, 7]:

$$K_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \tag{2.4}$$

$$K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}. \tag{2.5}$$

For the neighborhood samples p_{ij}, the gradients are

$$G_x = -p_{00} + p_{02} - 2p_{10} + 2p_{12} - p_{20} + p_{22}, \tag{2.6}$$

$$G_y = -p_{00} - 2p_{01} - p_{02} + p_{20} + 2p_{21} + p_{22}. \tag{2.7}$$

The coefficients are limited to $\{-2, -1, 0, +1, +2\}$, so multiplication by two is implemented as a one-bit shift and no general multiplier is required. Multiplier-free Sobel arithmetic is also common in FPGA implementations [7, 11].

For unsigned 8-bit input pixels,

$$|G_x|_{\max} = |G_y|_{\max} = 4 \times 255 = 1020.$$

Therefore,

$$-1020 \leq G_x, G_y \leq 1020,$$

and signed 11-bit values are sufficient for both gradients.

2.6 Gradient Magnitude

The Euclidean gradient magnitude is

$$M_{L2} = \sqrt{G_x^2 + G_y^2},$$

but multiplication and square-root operations would increase hardware complexity. The project therefore uses the L_1 magnitude,

$$M = |G_x| + |G_y|. \tag{2.8}$$

This form requires only absolute-value operations and addition. Since each Sobel gradient has a maximum magnitude of 1020,

$$M_{\text{max}} = 2040,$$

so the magnitude is represented using 12 unsigned bits.

2.7 Edge Thresholding

The gradient magnitude is converted into a binary edge image using a fixed threshold T:

$$E(x, y) = \begin{cases} 255, & M(x, y) \geq T, \\ 0, & M(x, y) < T. \end{cases} \tag{2.9}$$

The current design uses

$$T = 100.$$

The comparison is inclusive; therefore, $M = 100$ is classified as an edge. The complete algorithm is consequently

$$\text{RGB} \rightarrow \text{Grayscale} \rightarrow \text{Noise} \rightarrow \text{Median} \rightarrow \text{Sobel} \rightarrow |G_x| + |G_y| \rightarrow \text{Threshold}.$$

2.8 Streaming Image-Processing Architecture

A streaming FPGA architecture accepts pixels in raster order and allows different processing stages to operate concurrently on different pixels. After pipeline fill, a properly designed datapath can accept one new valid pixel per clock cycle, provided timing constraints are satisfied.

Two concepts are important:

- **Latency:** the delay between acceptance of an input item and availability of its corresponding result.
- **Throughput:** the rate at which valid items can be accepted or produced after the pipeline is filled.

A valid signal is propagated with the pixel stream so that each stage can distinguish meaningful data from invalid cycles. Coordinate information must also remain synchronized with registered data because each 3×3 stage shifts the mathematical center relative to the newest input pixel.

For a 3×3 window whose newest sample is at (x, y), the center coordinate is $(x - 1, y - 1)$. Cascading two neighborhood stages therefore requires careful valid-region contraction and coordinate alignment. The exact implementation is described in Chapter 3.

2.9 Line Buffer and Sliding Window

A 3×3 neighborhood requires pixels from three adjacent rows and three adjacent columns, while a streaming interface normally supplies only one new pixel per valid clock cycle. A line buffer therefore stores the two previous image rows and provides

$$I(x, y-2), \qquad I(x, y-1), \qquad I(x, y)$$

for the current column. This requires approximately $2W$ 8-bit storage locations for an image width W, rather than storing the complete frame.

The three vertically aligned samples are then passed to a sliding-window generator. For each row, two previous horizontal samples are retained while the newest column is supplied by the current input. This forms the 3×3 neighborhood required by the processing stage. Such line-buffer and pixel-cache structures are widely used in streaming FPGA image-processing pipelines [7, 12].

Because the proposed pipeline contains both Median and Sobel neighborhood operations, two independent line-buffer/window stages are required. Their RTL organization is presented in Chapter 3.

2.10 AXI4-Lite Interface

AXI is part of the Arm AMBA family and provides standardized communication between processors and memory-mapped peripherals [4]. AXI4-Lite is intended for relatively low-bandwidth register and control access and transfers one data beat per transaction without burst support.

The interface uses five independent channels, summarized in Table 2.4.

Table 2.4: AXI4-Lite communication channels.

Channel	Direction	Function
AW — Write Address	Master → Slave	Transfers the write address
W — Write Data	Master → Slave	Transfers write data and byte strobes
B — Write Response	Slave → Master	Reports write completion and status
AR — Read Address	Master → Slave	Transfers the read address
R — Read Data	Slave → Master	Returns read data and response information

Each channel uses the VALID/READY handshake; a transfer occurs when both signals are asserted on a clock edge. AXI4-Lite also provides byte strobes for selective byte writes and response codes for successful or erroneous accesses.

In the proposed system, the accelerator appears to the ARM processor as a memory-mapped peripheral. The project-specific register map, transaction behavior, and wrapper implementation are described in Chapter 3.

2.11 AMD Kria KR260 FPGA SoC Platform

The target platform is the AMD Kria KR260 Robotics Starter Kit, based on the Kria K26 System-on-Module and a Zynq UltraScale+ MPSoC [5]. The device integrates two major domains:

- **Processing System (PS):** Arm Cortex-A53 processors, memory controllers, and standard peripherals.
- **Programmable Logic (PL):** configurable FPGA resources used to implement custom hardware.

Ubuntu Linux runs on the Cortex-A53 processors and performs file handling, accelerator configuration, MMIO access, status monitoring, and result retrieval. The Median–Sobel accelerator is implemented in the PL and is accessed through the AXI interconnect.

This PS–PL partition allows software to manage system-level control while repetitive neighborhood processing is implemented as dedicated hardware. The detailed Vivado integration and Linux-side control flow are presented in Chapter 3.

2.12 Related FPGA Image-Processing Work

Prior FPGA work covers Median filtering, Sobel edge detection, streaming image pipelines, and SoC integration, but the reported designs differ in device family, clock frequency, algorithm scope, interface, and verification methodology. The comparison in Table 2.5 is therefore intended as architectural context rather than a direct ranking.

Table 2.5: Selected related FPGA image-processing implementations.

Work	Reported implementation	Relevance to this project
Bevara and Sanki [6]	Parallel 3×3 Median architectures on Xilinx Virtex devices.	Shows the latency/resource trade-off of parallel Median-filter hardware.
Schiel and Bainbridge-Smith [7]	Multiplier-free Sobel processing with local pixel caching on a low-cost FPGA.	Supports shift-add Sobel arithmetic and streaming neighborhood generation.
Kumar et al. [11]	Verilog 3×3 Sobel for 64×64 grayscale images at 100 MHz.	Closely matches the base image size and Sobel arithmetic, but does not include Median preprocessing or PS–PL integration.
Qamar et al. [12]	Zynq UltraScale+ ZCU102 streaming lane-detection pipeline at 150 MHz.	Provides a same-family SoC reference, although the algorithm and datapath differ.
Alshemi et al. [9]	Physical Zynq-7000 lane-detection accelerator compared with a GPU.	Demonstrates the importance of board-level timing and power measurements.
Liao [13]	HLS denoising pipeline using DRAM-backed buffering and burst AXI transfers.	Motivates migration from per-pixel AXI4-Lite access toward DMA or AXI4-Stream.

Additional studies reinforce two design choices of this work. Baru and Deokar [8] show the resource cost of larger Median windows, while Wu [10] discusses the area–throughput trade-off between iterative and highly parallel Median architectures. Sharanya and Sampath [14] also report software/hardware differences in a Median-plus-edge flow, emphasizing the importance of matched arithmetic and border conventions.

2.13 Chapter Summary

This chapter introduced the image-processing and SoC concepts required by the proposed accelerator. The software preprocessing stage generates deterministic 8-bit grayscale input, while the hardware pipeline performs 3×3 Median filtering, 3×3 Sobel gradient calculation, L_1 magnitude computation, and fixed-threshold edge generation.

The chapter also summarized streaming line-buffer/window processing, AXI4-Lite communication, the KR260 PS–PL organization, and selected related FPGA implementations. The detailed RTL architecture and system integration are presented in Chapter 3.

Chapter 3

SYSTEM DESIGN AND IMPLEMENTATION

This chapter presents the hardware architecture and system integration of the proposed Median–Sobel image-processing accelerator. Phase 1 implements the image-processing algorithm as a standalone streaming RTL pipeline. Phase 2 wraps the verified core with frame memories, control logic, a finite-state machine, an arbiter, and an AXI4-Lite interface so that it can be controlled by the ARM Processing System of the AMD Kria KR260.

The chapter first describes the system architecture and Phase 1 datapath, followed by the Phase 2 control structure, memory organization, AXI interface, Vivado integration, address map, and Linux-side control flow.

3.1 System Architecture Overview

The target system combines the ARM Processing System (PS) and Programmable Logic (PL) of the Zynq UltraScale+ MPSoC on the AMD Kria KR260. Ubuntu Linux runs on the Cortex-A53 processors and is responsible for image loading, control commands, status monitoring, and result retrieval. The Median–Sobel accelerator is implemented in the PL and appears to software as an AXI4-Lite memory-mapped peripheral.

Figure 3.1 shows the system-level organization.

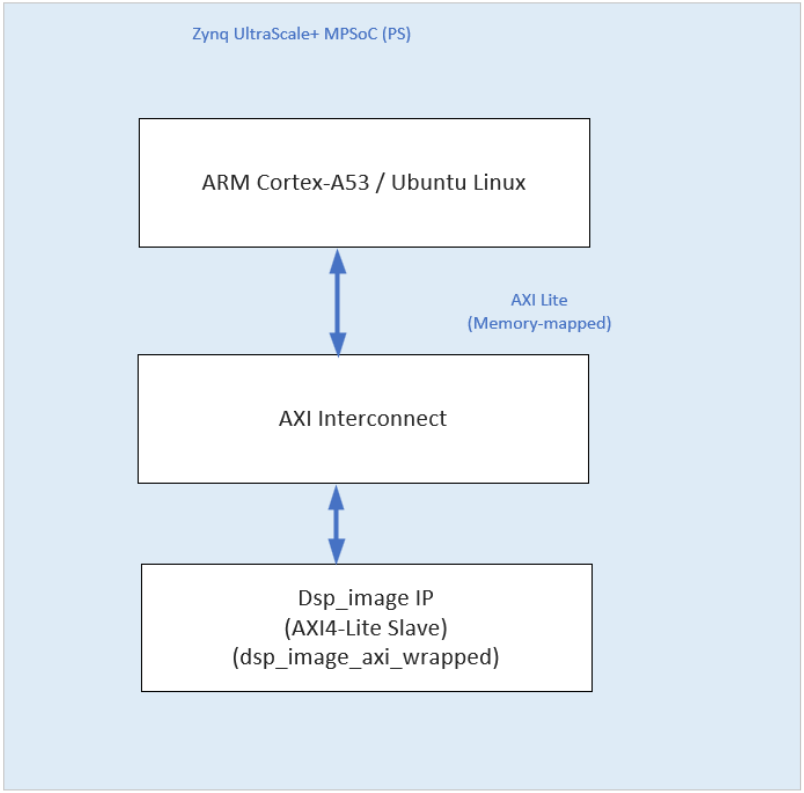


Figure 3.1: System-level architecture of the image-processing accelerator on the KR260 platform.

The implementation is divided into three logical layers:

1. **Phase 1 streaming DSP core**: Median–Sobel image processing;
2. **Phase 2 accelerator layer**: frame storage, arbitration, execution control, and state management;
3. **AXI4-Lite integration layer**: PS–PL communication and software access.

Table 3.1 summarizes the main design parameters.

Table 3.1: Principal design parameters.

Parameter	Current value / expression
Input pixel format	Unsigned 8-bit grayscale
Median window	3×3
Sobel window	3×3
G_x, G_y width	Signed 11-bit
Gradient magnitude width	Unsigned 12-bit
Threshold	100
Final pixel values	0 or 255
Input pixels	$W \times H$
Final valid pixels	$(W-4)(H-4)$
Current KR260 frame size	64×64
Input / output pixels	4,096 / 3,600
Phase 2 memory address width	12 bits
AXI4-Lite data / local address width	32 / 25 bits

3.2 Phase 1 Streaming DSP Core

Phase 1 implements the complete image-processing algorithm as a bus-independent streaming RTL core. It accepts an 8-bit grayscale pixel stream in raster order and produces an 8-bit binary edge stream.

Figure 3.2 shows the processing pipeline.

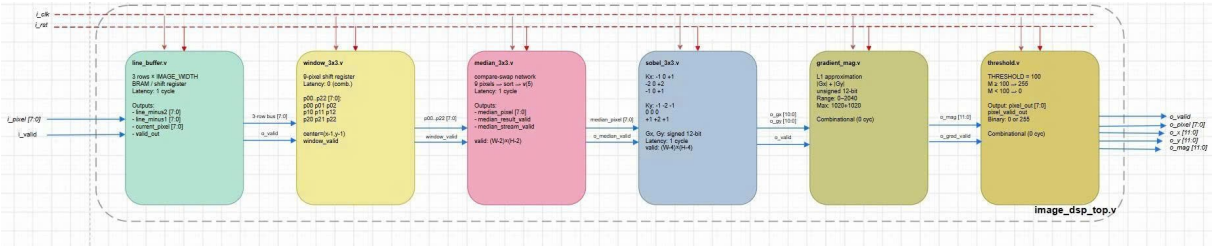


Figure 3.2: Architecture of the Phase 1 streaming image-processing pipeline.

The processing sequence is

$$\texttt{line_buffer} \rightarrow \texttt{window_3x3} \rightarrow \texttt{median_3x3} \rightarrow \texttt{line_buffer} \rightarrow \texttt{window_3x3} \rightarrow \texttt{sobel_3x3} \rightarrow \texttt{gradient_mag} \rightarrow \texttt{threshold}.$$

Two line-buffer/window pairs are required because both Median and Sobel are 3×3 neighborhood operations. The top level also propagates valid and coordinate information so that the final output is asserted only for

$$2 \le x \le W-3, \qquad 2 \le y \le H-3,$$

giving

$$N_{\text{out}} = (W-4)(H-4). \tag{3.1}$$

3.2.1 Line Buffer Architecture

The line buffer converts the raster stream into three vertically aligned samples corresponding to the current row and the two preceding rows. Figure 3.3 illustrates the architecture.

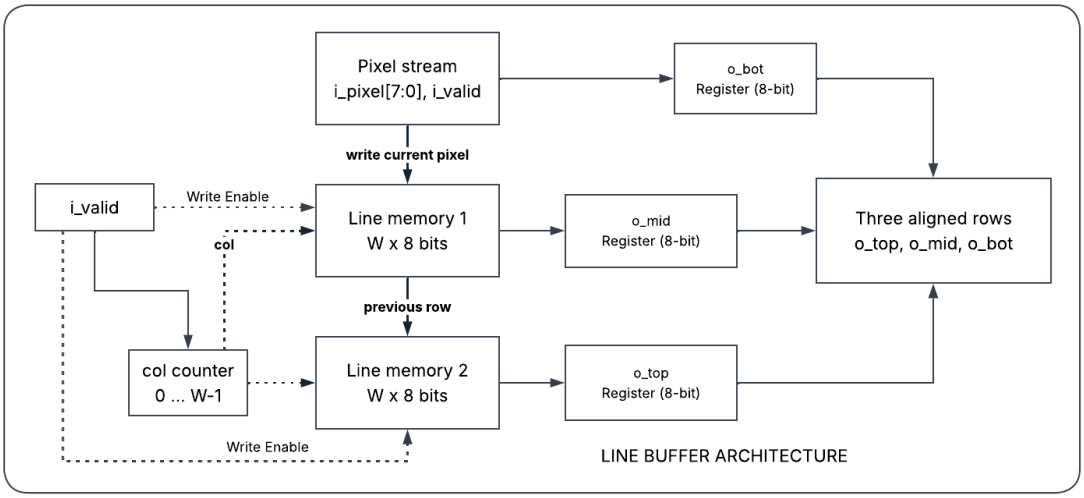


Figure 3.3: Pixel datapath architecture of the line-buffer module.

Two memories, `line1` and `line2`, store the previous two rows. For a valid input pixel at column c,

$$o_{\text{top}} = \texttt{line2}[c], \tag{3.2}$$

$$o_{\text{mid}} = \texttt{line1}[c], \tag{3.3}$$

$$o_{\text{bot}} = i_{\text{pixel}}, \tag{3.4}$$

while the row history is updated as

$$\texttt{line2}[c] \leftarrow \texttt{line1}[c], \tag{3.5}$$

$$\texttt{line1}[c] \leftarrow i_{\text{pixel}}. \tag{3.6}$$

The column counter advances only when input valid is asserted, so gaps in the stream do not disturb spatial alignment. One line-buffer stage requires approximately $2W$ bytes of 8-bit storage and introduces one registered clock cycle.

3.2.2 3×3 Sliding-Window Generator

The `window_3x3` module extends the three vertically aligned samples into a full 3×3 neighborhood. Figure 3.4 shows the implementation.

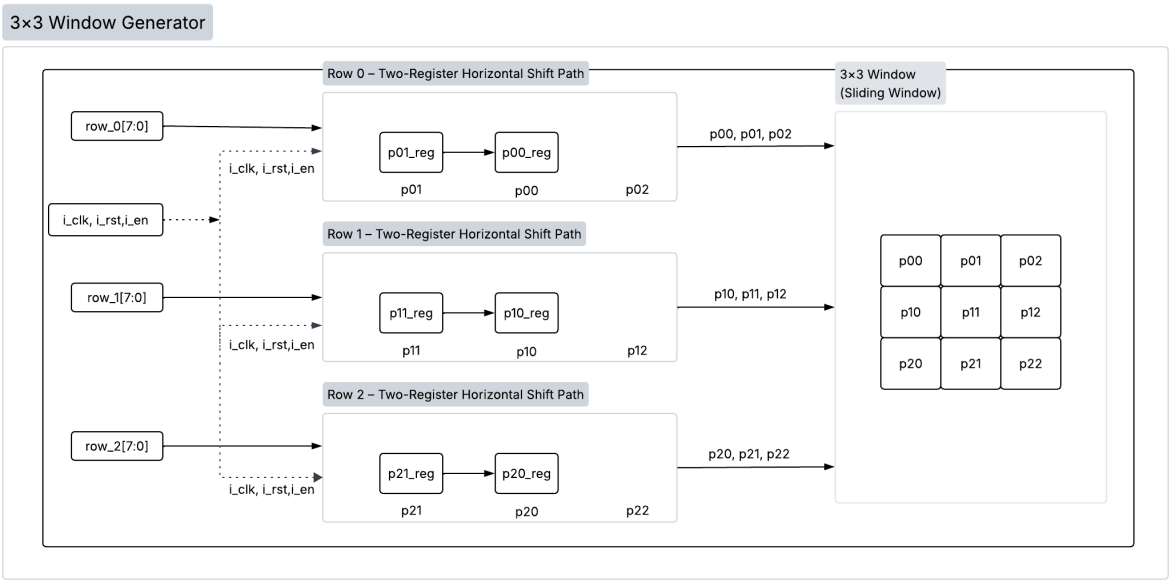


Figure 3.4: Architecture of the 3×3 sliding-window generator.

For each row, two registers retain the two previous horizontal samples and the newest sample forms the rightmost column. For example,

$$p01_{\text{reg}} \leftarrow i_{\text{row0}}, \tag{3.7}$$

$$p00_{\text{reg}} \leftarrow p01_{\text{reg}}, \tag{3.8}$$

giving

$$p_{00} = p00_{\text{reg}}, \qquad p_{01} = p01_{\text{reg}}, \qquad p_{02} = i_{\text{row0}}.$$

Rows 1 and 2 use the same structure. The registers shift only when `i_en` is asserted, preserving alignment when input-valid cycles contain gaps.

3.2.3 3×3 Median-Filter Architecture

The Median filter receives the nine samples from the first window generator and selects the fifth ordered value. Figure 3.5 shows the implemented compare-swap network.

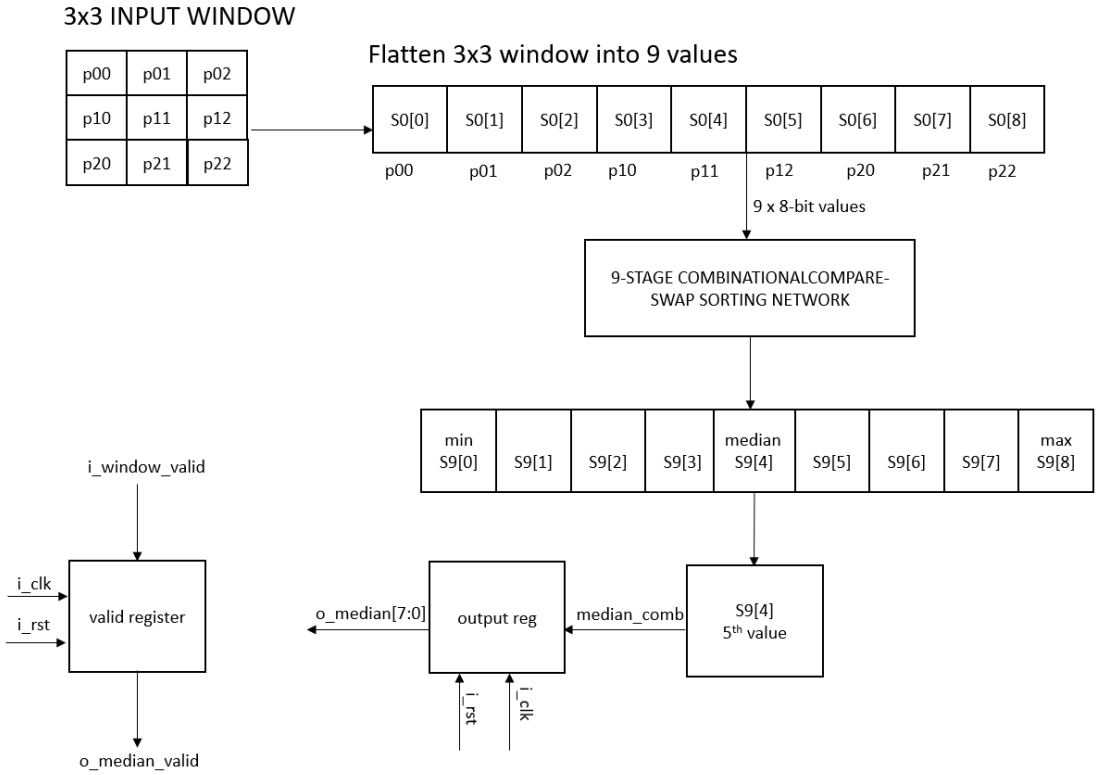


Figure 3.5: Architecture of the 3×3 Median filter using a nine-stage combinational compare-swap network.

The nine window samples are flattened into a nine-element array. An odd–even transposition sorting network alternates the comparison pairs

$$(0,1),(2,3),(4,5),(6,7)$$

and

$$(1,2),(3,4),(5,6),(7,8).$$

After nine combinational sorting stages,

$$s_9[0] \leq s_9[1] \leq \cdots \leq s_9[8],$$

and the Median is

$$\text{median} = s_9[4]. \tag{3.9}$$

The nine compare-swap stages are combinational rather than nine clock cycles. The final Median value and valid signal are registered once. This design therefore trades a deep combinational path for low registered latency, consistent with the area/timing trade-off discussed in prior Median-filter work [6, 10].

3.2.4 3×3 Sobel Datapath

The second line-buffer/window pair operates on the Median-filtered stream. The resulting neighborhood is processed by the Sobel datapath shown in Figure 3.6.

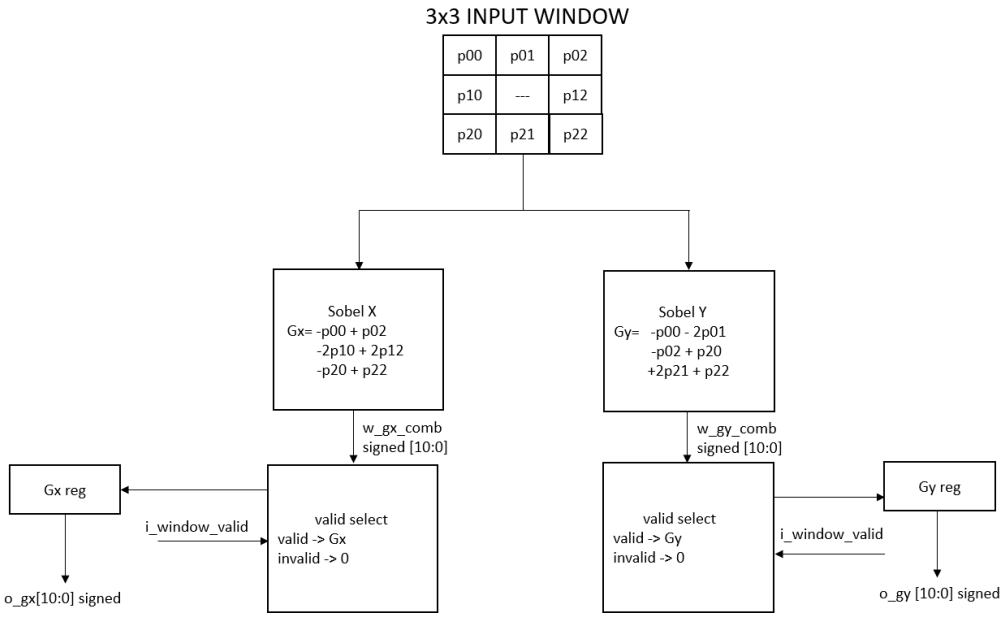


Figure 3.6: Datapath architecture of the 3×3 Sobel operator.

The gradients are

$$G_x = -p_{00} + p_{02} - 2p_{10} + 2p_{12} - p_{20} + p_{22}, \tag{3.10}$$

$$G_y = -p_{00} - 2p_{01} - p_{02} + p_{20} + 2p_{21} + p_{22}. \tag{3.11}$$

Multiplication by two is implemented as a one-bit left shift, so only signed addition, subtraction, and shifting are required [7]. Since

$$-1020 \leq G_x, G_y \leq 1020,$$

both gradients use signed 11-bit representations. The two paths operate in parallel and are captured into output registers, giving one registered cycle of Sobel latency.

3.2.5 Gradient Magnitude and Threshold Datapath

Figure 3.7 shows the final arithmetic stages.

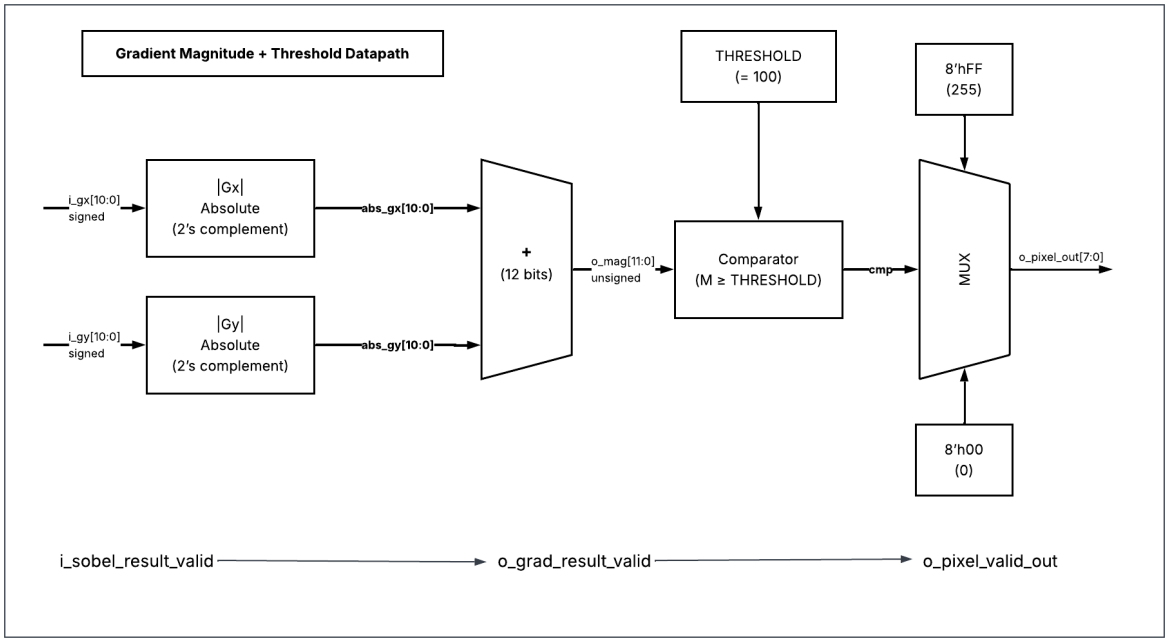


Figure 3.7: Datapath architecture of the gradient-magnitude and binary-threshold stages.

The gradient magnitude is computed combinationally as

$$M = |G_x| + |G_y|. \tag{3.12}$$

The 11-bit absolute values are extended to 12 bits before addition, allowing the maximum value

$$M_{\text{max}} = 2040.$$

The threshold stage is also combinational:

$$o_{\text{pixel}} = \begin{cases} 8'hFF, & M \geq 100, \\ 8'h00, & M < 100. \end{cases} \tag{3.13}$$

Neither magnitude nor threshold introduces an additional registered stage after the Sobel output register.

3.2.6 Registered Latency and Throughput Summary

Table 3.2 summarizes the registered latency of the Phase 1 pipeline.

Table 3.2: Registered latency of the Phase 1 streaming datapath.

Stage	Registered latency	Accept rate	Comment
Line Buffer 1	1 clk	1 px/clk	Two-row history
Window Generator 1	0 clk	1 px/clk	Combinational newest column
Median 3×3	1 clk	1 window/clk	Nine combinational sorting stages
Line Buffer 2	1 clk	1 px/clk	Operates on Median stream
Window Generator 2	0 clk	1 px/clk	Combinational newest column
Sobel 3×3	1 clk	1 window/clk	Registered G_x, G_y
Magnitude + Threshold	0 clk	1 result/clk	Combinational
Total registered datapath latency	**4 clk**	–	Excludes neighborhood fill and frame traversal

The four-cycle value is not the full-frame START-to-DONE latency. Phase 2 must still feed all WH input pixels through the core. For the 64×64 configuration at 100 MHz, input feeding alone has the lower bound

$$T_{\text{feed,min}} = \frac{4096}{100 \times 10^6} = 40.96\ \mu\text{s}. \tag{3.14}$$

The actual interval must be larger because it also includes preparation, synchronous memory response, pipeline fill/flush, final memory commit, and the transition to READ. Chapter 4 uses this lower bound to check the consistency of the software timing experiment.

3.3 Phase 2 Structured Accelerator

Phase 2 adds frame storage, processor-controlled execution, memory ownership, error handling, and bus access around the Phase 1 streaming core. Figure 3.8 shows the internal architecture.

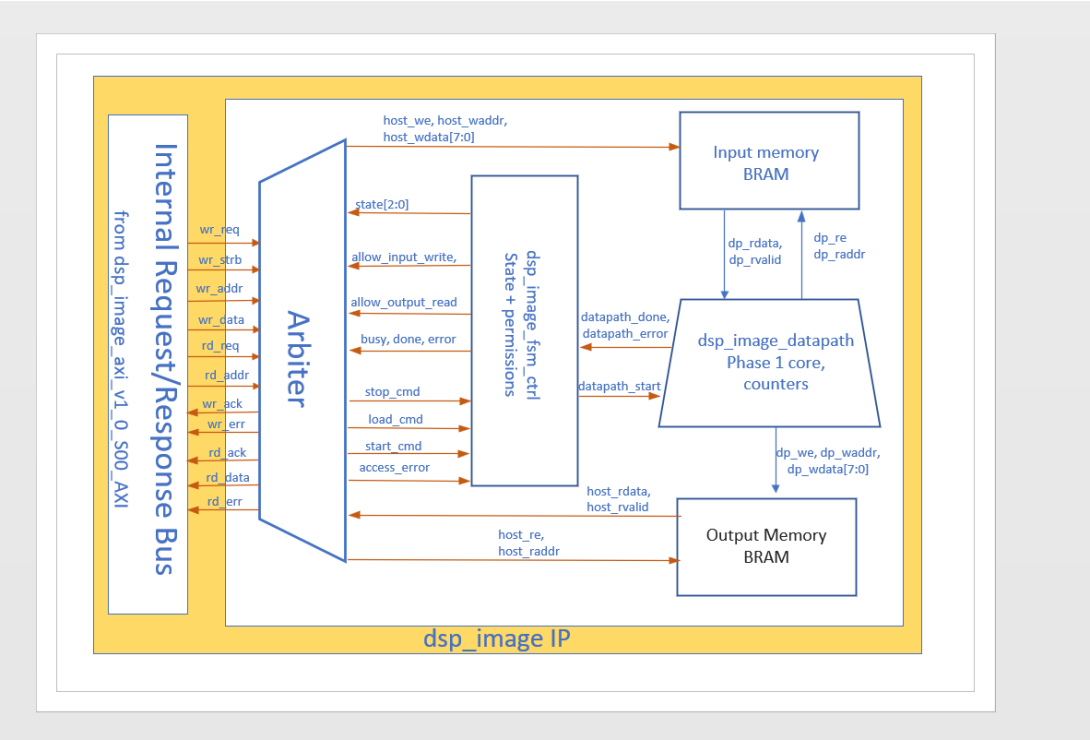


Figure 3.8: Internal architecture of the Phase 2 image-processing accelerator.

The principal modules are summarized in Table 3.3.

Table 3.3: Responsibilities of the Phase 2 modules.

Module	**Responsibility**
Arbiter	Decodes host requests, verifies legality, generates commands, and returns ACK/ERR responses
FSM controller	Defines legal operating states and memory-access permissions
Input memory	Stores the complete noisy grayscale input frame
Datapath	Reads the frame, resets/drives Phase 1, counts transactions, and stores valid results
Output memory	Stores the dense final valid-region output pixels

The Phase 2 control interface uses active-low synchronous reset `i_rst_n`; the active-high reset expected by Phase 1 is generated locally in the datapath wrapper.

3.3.1 Memory Ownership and Frame Memories

The system separates loading, execution, and result retrieval so that software and hardware do not write the same memory simultaneously.

Table 3.4: Memory ownership by accelerator state.

State	Input Memory	Output Memory
IDLE	No host write	No host read
LOAD	Host writes	No host read
EXEC	Datapath reads	Datapath writes
READ	No host write	Host reads
ERROR	Access disabled	Access disabled

Both memories are 8-bit arrays inferred as Block RAM using the `ram_style="block"` attribute. With

$$\texttt{MEM_ADDR_W} = 12,$$

each memory provides $2^{12} = 4096$ locations. The Input Memory uses a host-write port and synchronous datapath-read port; the Output Memory uses a datapath-write port and synchronous host-read port.

For the implemented 64×64 configuration,

$$N_{\text{input}} = 4096, \qquad N_{\text{output}} = (64-4)^2 = 3600,$$

so both frames fit within the 4096-entry memories.

3.3.2 Phase 2 Datapath Controller

The `dsp_image_datapath` module converts frame-memory transactions into the streaming interface expected by Phase 1. It tracks three counters:

- `requests_sent`: Input Memory read requests issued;
- `inputs_fed`: valid memory responses delivered to Phase 1;
- `outputs_captured`: valid Phase 1 outputs stored.

The required counts are

$$N_{\text{input}} = WH, \tag{3.15}$$

$$N_{\text{output}} = (W-4)(H-4). \tag{3.16}$$

The internal execution sequence is

$$\texttt{ST_IDLE} \rightarrow \texttt{ST_PREP} \rightarrow \texttt{ST_RUN} \rightarrow \texttt{ST_COMMIT} \rightarrow \texttt{ST_DONE},$$

with an additional `ST_ERROR` state.

During `ST_PREP`, the Phase 1 line-buffer, coordinate, and pipeline state is reset for one cycle:

$$\texttt{core_rst} = \neg\texttt{i_rst_n} \vee (\texttt{state} = \texttt{ST_PREP}). \tag{3.17}$$

During `ST_RUN`, sequential memory responses are fed to Phase 1 and each valid output is written densely to Output Memory. Dense output index k corresponds to

$$x = 2 + (k \bmod (W - 4)), \tag{3.18}$$

$$y = 2 + \left\lfloor \frac{k}{W - 4} \right\rfloor. \tag{3.19}$$

After the final output is presented to the memory interface, `ST_COMMIT` provides an additional cycle so that the registered write is completed before DONE is asserted.

3.3.3 Top-Level Control FSM

The system FSM controls legal software commands and memory-access permissions. Figure 3.9 shows the state transitions.

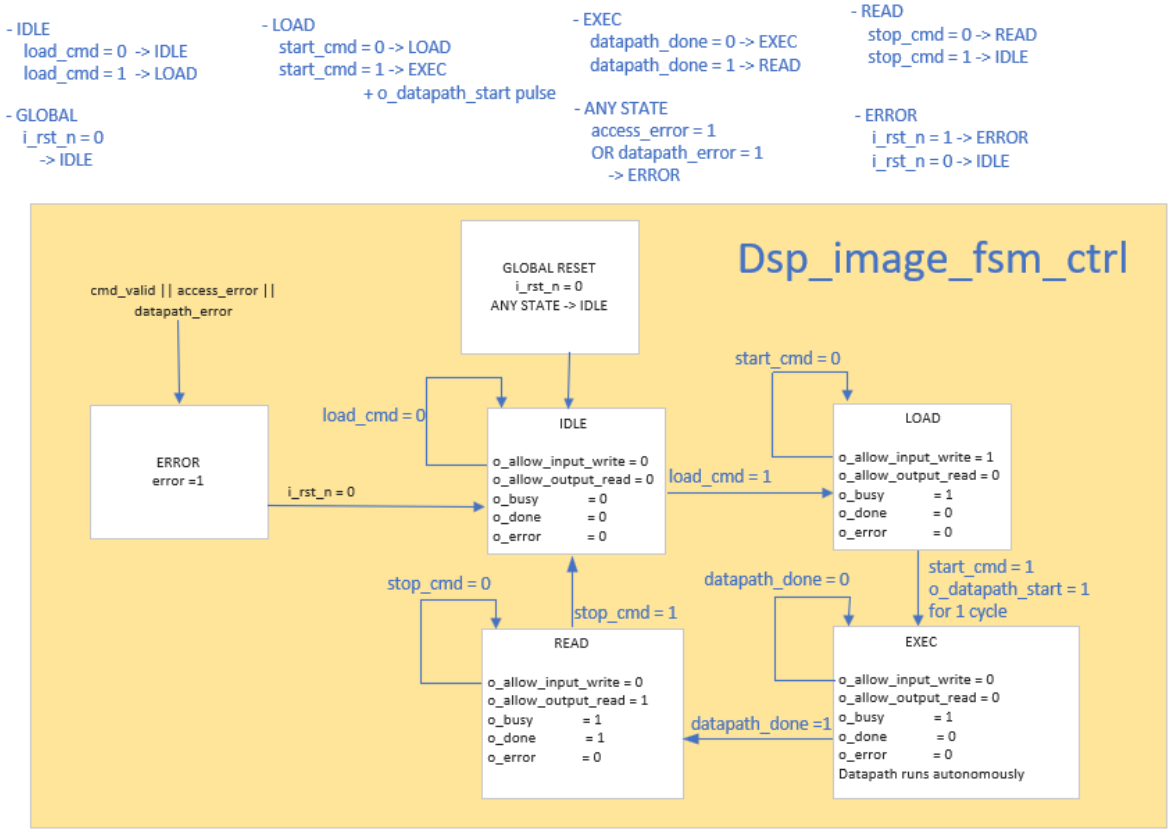


Figure 3.9: Finite-state machine of the Phase 2 accelerator controller.

The normal sequence is

$$\texttt{IDLE} \rightarrow \texttt{LOAD} \rightarrow \texttt{EXEC} \rightarrow \texttt{READ} \rightarrow \texttt{IDLE}.$$

Table 3.5 summarizes the state outputs.

Table 3.5: Outputs generated by the Phase 2 control FSM.

State	Input write	Output read	Busy	Done	Error
IDLE	0	0	0	0	0
LOAD	1	0	1	0	0
EXEC	0	0	1	0	0
READ	0	1	1	1	0
ERROR	0	0	0	0	1

START is legal only from LOAD and generates a one-cycle datapath-start pulse. ERROR is sticky and is entered on access, datapath, or invalid-command errors; reset is required to return to IDLE.

3.3.4 Arbiter and Internal Request/Response Bus

The AXI protocol terminates in the AXI wrapper, while the accelerator core uses a simpler internal request/response interface. The Arbiter

- decodes register and memory addresses;
- checks alignment, byte strobes, ranges, and FSM permissions;
- generates LOAD, START, and STOP command pulses; and
- returns ACK or ERR responses.

Input-memory writes are accepted only in LOAD and output-memory reads only in READ. Illegal accesses raise `access_error` and force the FSM into its sticky ERROR state.

3.4 AXI4-Lite Interface Implementation

The accelerator is packaged as a custom AXI4-Lite peripheral. Its main hierarchy is

```
dsp_image_axi_v1_0
  +-- dsp_image_axi_v1_0_S00_AXI
        +-- dsp_image_top
```

```
+-- dsp_image_arbiter
+-- dsp_image_fsm_ctrl
+-- dsp_image_input_memory
+-- dsp_image_datapath
+-- dsp_image_output_memory
```

The interface uses a 32-bit data bus and 25-bit local address bus, giving a 32-MiB local aperture. The modified slave logic supports independent AW/W acceptance, internal completion before response, response backpressure, and propagation of internal errors.

Successful requests return

$$\texttt{OKAY} = 2'b00,$$

while invalid operations return

$$\texttt{SLVERR} = 2'b10.$$

Each 8-bit pixel occupies one 32-bit address slot:

$$A(k) = A_{\text{base}} + 4k. \tag{3.20}$$

CTRL and input-pixel writes require

$$\texttt{WSTRB} = 4'b0001.$$

The present design uses AXI4-Lite only; AXI DMA and AXI4-Stream are not used.

3.5 Accelerator Register and Memory Map

Table 3.6 lists the local accelerator address map.

Table 3.6: Logical address map of the image-processing accelerator.

Offset	Name	Function
0x00000000	CTRL	Command register
0x00000004	STATUS	FSM and status bits
0x00000008	INFO_WIDTH	Configured image width
0x0000000C	INFO_HEIGHT	Configured image height
0x00000010	INFO_THRESHOLD	Configured threshold
0x00100000	INPUT_BASE	Input pixel memory
0x01100000	OUTPUT_BASE	Dense output pixel memory

The command values are shown in Table 3.7.

Table 3.7: CTRL register commands.

Value	Command	Legal state
`0x01`	LOAD	IDLE
`0x02`	START	LOAD
`0x04`	STOP	READ

Table 3.8 defines the status bits.

Table 3.8: STATUS register bit definitions.

Bit	Meaning
0	IDLE state
1	LOAD state
2	EXEC state
3	READ state
4	BUSY
5	DONE
6	ERROR

The read-only information registers report

$$\texttt{INFO_WIDTH} = 64, \qquad \texttt{INFO_HEIGHT} = 64, \qquad \texttt{INFO_THRESHOLD} = 100.$$

3.6 KR260 Vivado Integration

The packaged AXI4-Lite peripheral is integrated with the Zynq UltraScale+ Processing System, AXI interconnect, and processor-system reset network. Figure 3.10 shows the implemented Vivado design.

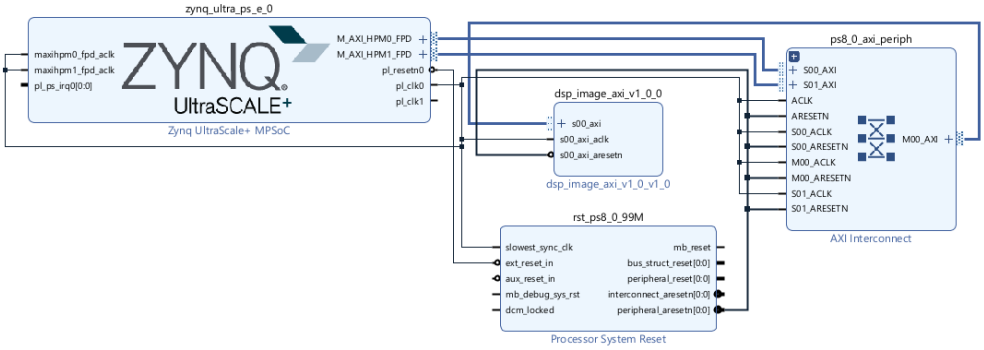


Figure 3.10: Vivado block design used to integrate the custom accelerator with the KR260 Processing System.

The PS master communicates with the accelerator through the AXI interconnect. The PS-generated PL clock drives the custom IP, and the associated reset block generates the AXI/peripheral reset signals. This organization follows the heterogeneous PS–PL architecture of the KR260 [5].

3.6.1 Physical Address Assignment

Vivado assigns the custom IP a 32-MiB physical aperture from

`0xA0000000`

to

`0xA1FFFFFF`.

Table 3.9 lists the relevant physical addresses.

Table 3.9: Physical KR260 addresses of the accelerator interface.

Function	Physical address
CTRL	`0xA0000000`
STATUS	`0xA0000004`
INFO_WIDTH	`0xA0000008`
INFO_HEIGHT	`0xA000000C`
INFO_THRESHOLD	`0xA0000010`
INPUT_BASE	`0xA0100000`
OUTPUT_BASE	`0xA1100000`

Input and output pixel addresses are therefore

$$A_{\text{input}}(k) = \texttt{0xA0100000} + 4k, \tag{3.21}$$

$$A_{\text{output}}(k) = \texttt{0xA1100000} + 4k. \tag{3.22}$$

3.7 Linux-Side Accelerator Control

Ubuntu Linux controls the accelerator through MMIO using `/dev/mem` and `mmap()` [5]. Register reads use 32-bit accesses, while CTRL and input-pixel writes use aligned 8-bit stores so that only the least-significant AXI byte lane is enabled.

The normal execution sequence is

1. read INFO and STATUS;
2. issue LOAD;
3. write $W \times H$ input pixels;
4. issue START;
5. poll STATUS until READ/DONE;
6. read $(W-4)(H-4)$ dense outputs;
7. issue STOP to return to IDLE.

The corresponding state progression is

$$\texttt{IDLE} \rightarrow \texttt{LOAD} \rightarrow \texttt{EXEC} \rightarrow \texttt{READ} \rightarrow \texttt{IDLE}.$$

A separate native ARM C program implements the same Median–Sobel algorithm entirely on the Cortex-A53 and is used as an independent software baseline in Chapter 4.

The current pixel transfers use individual AXI4-Lite transactions. This is suitable for validation but less efficient than burst-oriented AXI or DMA [4, 13]. The system-level performance consequence is discussed in Chapter 4.

Table 3.10 summarizes the main software and data artifacts.

Table 3.10: Software and data artifacts used in the verification and benchmark flow.

Artifact	Execution environment	Purpose
Python golden model	Host/KR260 Python	Preprocessing, reference generation, and comparison
`noisy.hex` / `expected.hex`	File interface	Reproducible input and golden-output exchange
Linux MMIO control program	KR260 Cortex-A53 / Ubuntu	LOAD, input writes, START, polling, output readback, STOP
Native ARM C DSP baseline	KR260 Cortex-A53 / Ubuntu	Same Median–Sobel algorithm without FPGA acceleration
Benchmark framework	KR260 Python/Flask	Repeated correctness and timing collection

3.8 Design Summary

The proposed accelerator is organized as a two-phase hardware system. Phase 1 implements the Median–Sobel algorithm as a streaming pipeline with two line-buffer/window pairs, a combinational Median sorting network, signed Sobel arithmetic, L_1 gradient magnitude, and binary thresholding. Its registered datapath latency is four clocks, while full-frame execution additionally depends on neighborhood fill, frame traversal, memory response, and completion overhead.

Phase 2 adds frame memories, datapath control, state-based ownership, error handling, and processor-visible access. The design is exposed through a 32-bit AXI4-Lite interface and integrated with the KR260 Processing System. Ubuntu software loads the input image, controls execution, reads the dense output, and provides the interface used for the verification and benchmarking presented in Chapter 4.

Chapter 4

VERIFICATION AND EVALUATION

This chapter evaluates the proposed Median–Sobel accelerator through RTL verification, integrated simulation, post-implementation analysis, and physical execution on the AMD Kria KR260. The results are reported according to their verification level: simulation correctness, static timing closure, and physical-board correctness are treated as separate forms of evidence. In particular, successful RTL simulation and positive post-route timing slack are not interpreted as proof of bit-exact behavior on the physical FPGA.

The RTL regressions pass at all tested levels, and the implemented design meets the 100-MHz timing constraint. On the KR260, the native ARM C implementation is bit-exact across repeated runs, while the FPGA implementation still exhibits one intermittent but spatially repeatable mismatch. A userspace START-to-DONE timing value was also recorded, but it is rejected as a validated FPGA latency because it violates the architectural cycle-count lower bound derived in Chapter 3.

4.1 Verification Methodology

The Python implementation is used as the numerical reference. Verification progresses from isolated RTL blocks to the complete streaming pipeline, Phase 2 accelerator, AXI4-Lite wrapper, post-route implementation, and finally the physical KR260 system. Table 4.1 summarizes the verification hierarchy.

Table 4.1: Verification levels used in the project.

Level	Main evidence	Target
Python reference	Golden-model scripts and reference HEX files	Numerical reference
Module-level RTL	Self-checking Verilog testbenches	Local functionality
Phase 1 integration	Full-frame RTL simulation against Python	Streaming DSP correctness
Phase 2 integration	Memory, datapath, FSM, arbiter, and multi-frame tests	Accelerator control/storage
AXI4-Lite integration	End-to-end bus-level simulation	PS–PL transactions
Post-implementation	Routing, utilization, timing, DRC, and power reports	Physical implementation
KR260 hardware	Linux MMIO, ARM C baseline, FPGA output comparison, repeated runs	Real-board behavior

For an image of width W and height H,

$$N_{\text{input}} = WH, \qquad N_{\text{output}} = (W-4)(H-4). \tag{4.1}$$

For the 64×64 hardware configuration, this corresponds to 4,096 input pixels and 3,600 valid output pixels.

4.2 Module-Level Verification

The principal datapath and control blocks were verified independently before system integration. Table 4.2 consolidates the standalone evidence that was previously discussed module by module.

Table 4.2: Module-level RTL verification summary.

Module	Main coverage	Checks	Result
Line Buffer	Row delays, valid gating, column wraparound, reset behavior	186	PASS
Window Generator	Shift enable, register retention, p_{00}–p_{22} ordering	1,071	PASS
Median Filter	Ordered/repeated values, impulse patterns, reset, valid propagation, deterministic random windows	618	PASS
Sobel + Gradient	Signed gradients and magnitude compared in the integrated Phase 1 reference flow	–	Integration covered
Threshold	Boundary cases 99/100/101 and exhaustive 12-bit input space	8,204	PASS

The Median test confirms correct selection of the fifth ordered value from the nine-input compare-swap network. The threshold regression confirms the inclusive condition $M \geq 100$. Sobel and gradient magnitude are strongly covered by the integrated golden-model comparison, although a separately preserved standalone Sobel regression is not available. This limitation is reported explicitly rather than presented as independent unit-test coverage.

4.3 Phase 1 End-to-End Verification

The complete streaming pipeline was compared against the Python golden model at three image resolutions. The testbench checks final pixel values, gradient magnitude, coordinates, and output count. Results are summarized in Table 4.3.

Table 4.3: Phase 1 end-to-end simulation results.

Resolution	Input pixels	Valid outputs	Mismatches	Status
64×64	4,096	3,600	0	PASS
512×512	262,144	258,064	0	PASS
640×480	307,200	302,736	0	PASS

All three resolutions are bit-exact relative to Python. A consecutive-frame 512×512 simulation also completed with zero errors, confirming that the streaming state is correctly reinitialized between frames in RTL simulation.

4.4 Phase 2 and AXI4-Lite Verification

Phase 2 integrates frame memories, the datapath controller, system FSM, arbiter, and the verified streaming core. Table 4.4 summarizes the principal regressions.

Table 4.4: Phase 2 simulation regression results.

Test	**Checks**	**Result**
Input memory	12,434	0 errors
Output memory	7,408	0 errors
Datapath, 512×512	516,133	0 errors; 262,144 inputs and 258,064 outputs
FSM	28	0 errors
Arbiter	40	0 errors
Top-level, 512×512	–	PASS
Top-level, 640×480	–	PASS; 302,736 dense outputs matched
Top-level two frames, 512×512	516,136	0 errors; 524,288 inputs and 516,128 outputs

The end-to-end 64×64 AXI regression then exercised register accesses, input writes, accelerator execution, output reads, backpressure, and error handling across two frames. It completed 8,199 AXI writes, 8,580 AXI reads, and 24,041 checks with zero errors. An illegal write returned `SLVERR`, entered the sticky ERROR state, and was successfully cleared by active-low reset. These results validate the simulated control and communication path but do not by themselves establish physical-board correctness.

4.5 Post-Implementation Evaluation

The complete design was synthesized, placed, and routed for the AMD Kria K26 device. Implementation results are reported separately from RTL functional verification.

4.5.1 Routing and Resource Utilization

All 6,689 routable nets were fully routed with zero routing errors. The reports contain six `PDCN-1569`, one `RTSTAT-10`, and six `LUTAR-1` methodology warnings. These warnings did not prevent routing or bitstream generation, but they are retained as implementation observations rather than claiming a warning-free design.

Table 4.5: Post-placement resource utilization of the implemented design.

Resource	Used	Available	Utilization
CLB LUTs	3,560	117,120	3.04%
LUT as Logic	3,341	117,120	2.85%
LUT as Memory	219	57,600	0.38%
CLB Registers	3,406	234,240	1.45%
CARRY8	68	14,640	0.46%
Block RAM Tiles	2	144	1.39%
UltraRAM	0	64	0.00%
DSP slices	0	1,248	0.00%

The current 64×64 implementation occupies only a small fraction of the K26 PL and uses no DSP slices. The latter is consistent with the shift-add Sobel arithmetic. These utilization values apply to the implemented small-frame configuration and should not be extrapolated directly to full-frame 512×512 or 640×480 Phase 2 memories.

4.5.2 Timing and Critical Path

The implemented PL clock is constrained to 100 MHz (10.000 ns). Post-route timing results are shown in Table 4.6.

Table 4.6: Post-route timing results at 100 MHz.

Metric	Result
Worst Negative Slack (WNS)	$+0.322$ ns
Total Negative Slack (TNS)	0.000 ns
Worst Hold Slack (WHS)	$+0.013$ ns
Total Hold Slack (THS)	0.000 ns
Setup failing endpoints	0
Hold failing endpoints	0
Unconstrained internal endpoints	0
Timing status	MET

The worst setup path runs from the first window generator to the registered Median output and therefore identifies the combinational Median network as the principal timing bottleneck.

Table 4.7: Critical setup path through the Median-filter network.

Parameter	Result
Source	`u_win1/p00_reg_reg[0]`
Destination	`u_median/o_median_reg[4]`
Data-path delay	9.465 ns
Logic delay	4.478 ns (47.3%)
Routing delay	4.987 ns (52.7%)
Logic levels	24
Worst setup slack	$+0.322$ ns
Status	MET

The positive slack means that static timing analysis does not prove that this path causes the later physical mismatch. It is therefore reported as the dominant implementation bottleneck and a candidate for further investigation, not as a confirmed timing failure.

4.5.3 Power Estimation

Vivado estimates total on-chip power at 2.737 W, consisting of 2.437 W dynamic and 0.301 W static power, with an estimated junction temperature of 31.4°C. The report confidence is Medium and no detailed simulation activity file was used. These values are therefore implementation estimates rather than direct KR260 board-power measurements.

4.6 KR260 Hardware Evaluation

The final bitstream was loaded on the KR260 through the Linux FPGA Manager, which reported the PL state as `operating`. The Linux control program accesses the accelerator through `/dev/mem` and `mmap()`; CTRL and input writes use aligned 8-bit stores to satisfy `WSTRB=0001`.

4.6.1 MMIO Bring-Up and Black-Frame Test

The initial read-only probe returned `INFO_WIDTH=64`, `INFO_HEIGHT=64`, `INFO_THRESHOLD=100`, and `STATUS=0x01` (IDLE). A LOAD command moved the accelerator to `0x12`, the complete 4,096-pixel input frame was written successfully, and START eventually produced `0x38` (READ+BUSY+DO
This verifies physical configuration, register access, input-memory transfer, execution control, and output-read access.

A deterministic all-zero frame was then processed. All 3,600 valid outputs were `0x00`, giving

$$3600/3600 \text{ outputs correct.} \tag{4.2}$$

The black-frame test confirms the basic physical execution path, although a constant image does not exercise the full data-dependent behavior of the Median and Sobel datapaths.

4.6.2 Golden-Model Consistency Check

Before hardware comparison, the web benchmark's Python golden output was compared numerically with the original `expected.hex`. Both contain 4,096 entries and

$$N_{\text{difference}} = 0. \tag{4.3}$$

Figure 4.1 shows the result. This confirms that later FPGA mismatches are not caused by disagreement between two software reference models.

```
if len(web) == len(old) and not errors:
    print("WEB GOLDEN NUMERICALLY == ORIGINAL EXPECTED: PASS")
else:
    print("WEB GOLDEN NUMERICALLY DIFFERENT: FAIL")
PY
WEB pixels = 4096
OLD pixels = 4096
NUMERIC DIFFERENCES = 0
WEB GOLDEN NUMERICALLY == ORIGINAL EXPECTED: PASS
ubuntu@KR260-02:~/TueNT/image_dsp/image_dsp_standalone_benchmark/image_dsp_standalone_benchmark$ echo "FPGA:"
sed -n '2307p' "$LATEST/fpga_output.hex"
```

Figure 4.1: Numerical validation of the web-generated Python golden output against the original `expected.hex`; all 4,096 entries match.

4.6.3 Native ARM C Baseline

The same Median–Sobel algorithm was implemented in native AArch64 C and compiled with `-O3`. The timer surrounds only the algorithmic kernel, excluding file I/O and process startup. Across 20 repeated executions, ARM C produced zero mismatches in every run.

Table 4.8: Native ARM C kernel timing over 20 executions.

Metric	**Result**
Mean	4.817457 ms
Median	4.820223 ms
Minimum	4.753018 ms
Maximum	4.918889 ms
Standard deviation	0.040068 ms
Correct runs	20/20

The small standard deviation indicates stable processor-side execution for the tested workload.

4.6.4 FPGA Timing Consistency and Repeated Benchmark

A userspace timer was added inside the Linux control program to measure from immediately before the START MMIO write until READ/DONE was observed by tight status polling. Across 20 runs it recorded a mean of 0.023008 ms, with values from 0.022670 to 0.023561 ms. These measurements are retained as diagnostic software-timer observations only.

The current Phase 2 architecture must feed all 4,096 pixels to the streaming core. Even under the ideal assumption of one accepted pixel per 100-MHz PL cycle,

$$T_{\text{feed,min}} = \frac{4096}{100 \times 10^6} = 40.96\ \mu\text{s}. \tag{4.4}$$

The actual START-to-DONE interval must be longer because of `ST_PREP`, synchronous Input Memory response latency, pipeline fill/flush, final output commit, and the transition into READ. Therefore, the recorded 22.670–23.561 μs interval cannot represent the intended FPGA START-to-DONE latency under the stated 100-MHz clock. A hardware cycle counter or ILA measurement is required before a kernel-speedup claim is valid.

The same noisy 64×64 frame was nevertheless executed 20 times to evaluate correctness. Figure 4.2 preserves the original terminal output as experimental evidence. Its FPGA timing and derived "speedup" columns are not used as validated performance results.

```
PY      )   f"GOLDEN={expected:02X}"keymost_common():ms")or r in rows)}")
 RUN  ARM_ERR  FPGA_ERR     ARM_MS    FPGA_MS    SPEEDUP
   1        0         1   4.838848   0.023500     205.91
   2        0         1   4.753018   0.022860     207.92
   3        0         1   4.820289   0.023000     209.58
   4        0         1   4.810548   0.022831     210.70
   5        0         1   4.883899   0.022721     214.95
   6        0         1   4.832798   0.023130     208.94
   7        0         1   4.827118   0.023061     209.32
   8        0         1   4.839158   0.023561     205.39
   9        0         0   4.836449   0.022670     213.34
  10        0         1   4.828738   0.022870     211.14
  11        0         0   4.754307   0.022850     208.07
  12        0         0   4.820158   0.022841     211.03
  13        0         1   4.790048   0.023320     205.41
  14        0         1   4.817519   0.022960     209.82
  15        0         1   4.762248   0.022831     208.59
  16        0         1   4.757307   0.023110     205.85
  17        0         1   4.823058   0.022770     211.82
  18        0         1   4.819408   0.022980     209.72
  19        0         0   4.815328   0.022781     211.37
  20        0         0   4.918889   0.023520     209.14

============================================================
SUMMARY
============================================================
Runs                  : 20
ARM runs with errors  : 0
FPGA runs with errors : 15

ARM C KERNEL
------------
Mean   : 4.817457 ms
Median : 4.820223 ms
Min    : 4.753018 ms
Max    : 4.918889 ms
Stddev : 0.040068 ms

FPGA INTERNAL START -> DONE
---------------------------
Mean   : 0.023008 ms
Median : 0.022915 ms
Min    : 0.022670 ms
Max    : 0.023561 ms
Stddev : 0.000265 ms

ARM C / FPGA SPEEDUP
--------------------
Mean   : 209.40x
Median : 209.45x
Min    : 205.39x
Max    : 214.95x
Stddev : 2.51x

FPGA MISMATCH FREQUENCY
-----------------------
15x : k=2306 x=28 y=40 FPGA=00 GOLDEN=FF
ubuntu@KR260-02:~/TueNT/image_dsp/image_dsp_standalone_benchmark/image_dsp_standalone_benchmark$
```

Figure 4.2: Twenty-run KR260 benchmark log. ARM C and FPGA correctness results are used directly; the displayed FPGA timing and derived speedup are retained only as diagnostic software-timer output because they fail the cycle-count consistency check.

ARM C remained bit-exact in all 20 runs. The FPGA produced five bit-exact runs and 15 runs containing exactly one mismatch, corresponding to a 75% failing-run rate. Since the FPGA kernel time is not yet validated, the previously computed 209.40× ratio is not reported as a project speedup. At system level, future measurements should distinguish kernel time from input transfer, control, and output transfer:

$$T_{\text{FPGA,E2E}} = T_{\text{input}} + T_{\text{control}} + T_{\text{kernel}} + T_{\text{output}}. \tag{4.5}$$

4.6.5 Analysis of the Intermittent FPGA Mismatch

Every failing run contains the same mismatch:

$$k = 2306, \qquad (x, y) = (28, 40), \qquad P_{\text{FPGA}} = \texttt{0x00}, \qquad P_{\text{golden}} = \texttt{0xFF}. \tag{4.6}$$

The failure is therefore *intermittent but spatially deterministic*: it does not occur in every run, but whenever a failure appears, it occurs at the same output location and with the same wrong value.

The local 5×5 noisy neighborhood is

$$\begin{bmatrix} \texttt{D6} & \texttt{BE} & \texttt{92} & \texttt{9C} & \texttt{B1} \\ \texttt{AB} & \texttt{90} & \texttt{91} & \texttt{87} & \texttt{A2} \\ \texttt{B8} & \texttt{A6} & \texttt{88} & \texttt{6A} & \texttt{8F} \\ \texttt{AC} & \texttt{8D} & \texttt{73} & \texttt{70} & \texttt{FF} \\ \texttt{7D} & \texttt{77} & \texttt{84} & \texttt{95} & \texttt{8C} \end{bmatrix}.$$

Recomputing the Median stage gives the Sobel input window

$$\begin{bmatrix} \texttt{A6} & \texttt{91} & \texttt{91} \\ \texttt{91} & \texttt{88} & \texttt{88} \\ \texttt{88} & \texttt{84} & \texttt{88} \end{bmatrix},$$

from which

$$G_x = -39, \qquad G_y = -65, \qquad M = |-39| + |-65| = 104. \tag{4.7}$$

Because $104 \geq 100$, the correct result is `0xFF`. Other locations with the same magnitude of 104, including $(18, 40)$ and $(49, 41)$, produce the correct FPGA result. A simple threshold-comparator error is therefore unlikely.

The post-route critical path passes through the Median compare-swap network with 9.465 ns data delay and $+0.322$ ns setup slack. The data-dependent and intermittent failure makes timing sensitivity a plausible hypothesis, but static timing analysis still reports no setup or hold violations. The Median network is therefore a candidate for investigation, not a confirmed root cause. Other possibilities include intermediate-data or valid/alignment behavior between the Median output and second window stage. Direct ILA observation of the Median output, second-window pixels, G_x, G_y, magnitude, valid, and coordinates around $(28, 40)$ is required for definitive localization.

4.7 Comparison with Related FPGA Work

The detailed literature comparison is presented in Table 2.5 of Chapter 2 and is not repeated here. The measured implementation results add three project-specific observations. First, the Sobel arithmetic uses no DSP slices, consistent with multiplier-free FPGA Sobel designs [7]. Second, the design meets its stated 100-MHz static timing constraint, but the Median network is the dominant critical path. Third, unlike studies reporting fully validated board-level latency and speedup [9], this project explicitly withholds a CPU–FPGA speedup claim until both physical

correctness and cycle-accurate FPGA timing are established. The current per-pixel AXI4-Lite path also motivates future migration toward burst or DMA-based transfer, as emphasized in high-throughput FPGA imaging work [13].

4.8 Verification and Evaluation Summary

Table 4.9 summarizes the final status without repeating the detailed evidence given above.

Table 4.9: Overall verification and evaluation summary.

Component / level	**Evidence**	**Status**
Module-level RTL	Line Buffer 186; Window 1,071; Median 618; Threshold 8,204 checks, all zero-error	PASS
Phase 1	64×64, 512×512, 640×480, zero mismatches	PASS
Phase 2	Memories, datapath, FSM, arbiter, single- and multi-frame tests	PASS in simulation
AXI4-Lite	Two-frame 64×64 regression, 24,041 checks, zero errors	PASS in simulation
Routing	6,689/6,689 nets fully routed, zero routing errors	PASS
Timing at 100 MHz	WNS $+0.322$ ns; WHS $+0.013$ ns; no failing endpoints	PASS
KR260 black frame	3,600/3,600 outputs correct	PASS
Python golden consistency	4,096/4,096 entries identical to original reference	PASS
KR260 ARM C	20/20 bit-exact; mean kernel time 4.817457 ms	PASS
KR260 FPGA noisy frame	5/20 bit-exact; 15/20 contain one mismatch at $(28, 40)$	INTERMITTENT
FPGA START-to-DONE timing	Recorded mean 0.023008 ms, below 0.04096 ms architectural lower bound	INVALID FOR LATENCY CLAIM
CPU–FPGA speedup	No validated FPGA kernel latency available	NOT VALIDATED

Overall, the algorithm and RTL architecture are functionally correct in simulation, the routed implementation meets its 100-MHz timing constraint, and PS–PL communication operates successfully on the KR260. The ARM C baseline is repeatable and bit-exact. However, the physical FPGA still exhibits one intermittent location-specific mismatch, and the current userspace timing measurement is inconsistent with the architecture. Therefore, the project should claim successful simulation verification, timing closure, and hardware integration, but not a complete physical-hardware correctness PASS or CPU–FPGA speedup until these two issues are resolved.

Chapter 5

CONCLUSION AND FUTURE WORK

This chapter summarizes the verified outcomes of the project, identifies the limitations that remain on the physical KR260 implementation, and defines the most important next steps. Detailed numerical evidence is kept in Chapter 4 and is referenced here rather than repeated.

5.1 Achievements

The project completed the full development path from a deterministic software reference to an FPGA SoC prototype. The main verified achievements are as follows.

- A deterministic Python reference implements grayscale preprocessing, fixed-seed salt-and-pepper noise, 3×3 Median filtering, 3×3 Sobel gradients, the integer L1 magnitude $|G_x| + |G_y|$, and thresholding. The final web-generated golden data were numerically identical to the original reference HEX file.

- The synthesizable Phase 1 RTL pipeline was verified with zero mismatches for 64×64, 512×512, and 640×480 images. The architecture combines two line-buffer/window stages, a combinational Median sorting network, multiplier-free Sobel arithmetic, and explicit cascaded valid-region contraction. These choices are consistent with established FPGA approaches to median filtering, local-window Sobel processing, and multiplier-free gradient computation [6, 7, 10].

- The Phase 2 accelerator integrates Input Memory, Output Memory, a datapath controller, FSM, Arbiter, and AXI4-Lite wrapper. The two-frame AXI regression completed 24,041 checks with zero errors, including illegal-access handling and reset recovery.

- The design was implemented for the AMD Kria KR260 [5]. Post-route timing closes at 100 MHz with WNS $+0.322$ ns and WHS $+0.013$ ns. The complete implemented PL design uses 3,560 CLB LUTs, 3,406 registers, two Block RAM tiles, and no DSP slices. Vivado estimates 2.737 W total on-chip power with Medium confidence; this remains an estimate rather than a board-level measurement.

- PS–PL control through AXI4-Lite [4] was demonstrated from Ubuntu Linux, including LOAD, frame writes, START, status polling, output readback, STOP, and error-state recovery.
- A separate optimized ARM C implementation provides an independent processor baseline. It achieved 20/20 bit-exact runs with a mean kernel execution time of 4.817457 ms.

Relative to prior work, the project is distinguished less by a single resource or latency number than by the combination of a cascaded Median–Sobel RTL pipeline, explicit bit-exact Python/RTL verification across multiple resolutions, a processor-controlled SoC wrapper, and physical-board debugging. Direct numerical ranking against other publications is intentionally avoided because the cited designs differ in FPGA technology, frequency, algorithm scope, memory interface, and validation method [12, 11, 9].

5.2 System Limitations

The most important limitation is physical FPGA correctness. For the repeated noisy-image benchmark, the ARM C implementation passes all 20 runs, while the FPGA passes 5/20. Each of the 15 failing FPGA runs contains the same single mismatch at dense output index 2306, corresponding to coordinate $(28, 40)$, where the golden output is `0xFF` and the observed FPGA value is `0x00`. The behavior is therefore intermittent but spatially deterministic. Static timing analysis still reports positive setup and hold slack, so the Median critical path is a candidate for investigation rather than a proven root cause.

A second limitation is the FPGA latency measurement. The userspace benchmark recorded a mean START-to-DONE value of 23.008 μs. However, the Phase 2 datapath must feed 4,096 input pixels and can accept at most one pixel per 100-MHz clock. Input feeding alone therefore requires at least

$$\frac{4096}{100 \times 10^6} = 40.96\ \mu\text{s}, \tag{5.1}$$

before preparation, synchronous memory response, pipeline flush, and commit overhead are added. The recorded 23.008 μs value is therefore inconsistent with the current architecture and is not used as a validated FPGA kernel latency. Consequently, the report does not claim the previously derived 209.40$\times$ CPU–FPGA ratio. A separate 64×64 one-pixel-per-clock Sobel implementation likewise reports a 4,096-cycle frame scale at 100 MHz, reinforcing this consistency check [11].

The system also remains limited by data movement. AXI4-Lite is intentionally simple and useful for register-level bring-up, but it does not support burst transfers [4]. Every pixel is currently transferred through an individual MMIO address slot, so system-level throughput is dominated by software-controlled transactions rather than the internal streaming datapath.

Burst-oriented AXI designs reported in recent FPGA imaging work demonstrate why DMA or AXI4-Stream is the natural next step for scaling [13].

Finally, the physical implementation is currently configured for 64×64 frames. Larger 512×512 and 640×480 cases are verified in RTL simulation but not implemented with equivalent full-frame on-chip memories on the KR260. Power is estimated by Vivado rather than measured electrically, and the Sobel/gradient block has strong integrated coverage but lacks a preserved standalone self-checking unit regression comparable to the Median and Threshold tests.

5.3 Future Work

Future development should prioritize correctness and measurement quality before further optimization.

1. **Localize the intermittent mismatch.** Add an ILA or equivalent debug instrumentation around the Median output, second sliding window, G_x, G_y, magnitude, valid signals, coordinates, and FSM state near $(28, 40)$. A controlled lower-frequency build can also test whether the failure rate is sensitive to timing margin.

2. **Add a hardware cycle counter.** Count PL clock cycles from accepted START to datapath completion and expose the count through a read-only register. This removes Linux scheduling and polling ambiguity and provides a valid basis for kernel-speedup calculation.

3. **Strengthen standalone Sobel verification.** Add a dedicated self-checking testbench covering positive and negative gradient extremes, signed 11-bit boundaries, magnitude range, valid alignment, and threshold-adjacent cases.

4. **Pipeline the Median network if required.** The Median compare-swap network is the critical post-route path. Additional registers could increase timing margin at the cost of latency and alignment complexity, a trade-off also seen in published Median architectures [6, 10].

5. **Replace per-pixel MMIO for frame data.** Retain AXI4-Lite for control/status, but move image data through AXI DMA or AXI4-Stream with DDR or tiled buffering. This follows the broader trend toward burst/streaming data movement in high-throughput FPGA imaging systems [13].

6. **Scale and re-benchmark.** After correctness is stable, evaluate larger physical images, report cycle-accurate FPGA kernel time separately from input/output transfer time, and perform a fair end-to-end comparison with the ARM C baseline. The Orange Pi demonstration can then reuse the same input, algorithm, compiler settings, and exported JSON/CSV format for a cross-platform CPU comparison.

5.4 Final Remarks

The project demonstrates a complete and verifiable FPGA SoC development flow for a classical image-processing workload: numerical modeling, streaming RTL design, modular and end-to-end simulation, AXI4-Lite integration, implementation analysis, Linux control, native ARM comparison, and physical-board execution. The strongest validated results are the bit-exact RTL regressions, successful 100-MHz timing closure, low use of dedicated arithmetic resources, and repeatable ARM C baseline. The physical FPGA still requires one correctness issue and one timing-measurement inconsistency to be resolved. Reporting these limitations explicitly provides a sounder engineering conclusion than presenting an unverified speedup, and it establishes a clear path toward a robust DMA-enabled, cycle-measured, larger-frame accelerator.

Bibliography

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.

[2] T. S. Huang, G. J. Yang, and G. Y. Tang, "A fast two-dimensional median filtering algorithm," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 27, no. 1, pp. 13–18, 1979, doi: 10.1109/TASSP.1979.1163188.

[3] International Telecommunication Union, *Recommendation ITU-R BT.601-7: Studio Encoding Parameters of Digital Television for Standard 4:3 and Wide-Screen 16:9 Aspect Ratios*, Mar. 2011.

[4] Arm Ltd., *AMBA AXI and ACE Protocol Specification*, document IHI 0022, including the AXI4 and AXI4-Lite protocol definitions.

[5] AMD, *Kria KR260 Robotics Starter Kit User Guide*, UG1092, Rev. 1.1, Apr. 24, 2024.

[6] V. Bevara and P. K. Sanki, "A new fast and efficient 2-D median filter architecture," *Sādhanā*, vol. 45, Art. no. 192, 2020, doi: 10.1007/s12046-020-01431-0.

[7] J. Schiel and A. Bainbridge-Smith, "Efficient edge detection on low-cost FPGAs," arXiv:1512.00504, 2015.

[8] V. B. Baru and C. S. Deokar, "VLSI implementation of image segmentation with resource optimized adaptive median filter," in *Proc. International Conference on VLSI, Communication & Instrumentation (ICVCI)*, pp. 14–17, 2011.

[9] M. Alshemi, S. Saif, and M. Taher, "Hardware acceleration of lane detection algorithm: A GPU versus FPGA comparison," in *Computer Science & Information Technology (CS & IT)*, pp. 191–201, 2022, doi: 10.5121/csit.2022.122215.

[10] J. Wu, "The Tiny Median Filter: A small size, flexible arbitrary percentile finder scheme suitable for FPGA implementation," arXiv:2412.05320, 2024.

[11] N. V. S. Kumar, N. S. Kallakuri, P. Chandana, and C. Raja, "Sobel edge detection algorithm using Verilog for 64 X 64 grayscale image," *International Journal of Research and Scientific Innovation*, vol. 12, no. 9, pp. 4247–4255, 2025, doi: 10.51244/IJRSI.2025.120800384.

[12] I. M. I. Qamar, S. Mahmoud, S. Megahed, M. Khaled, S. Hesham, A. Matar, S. Gebril, and M. M. A. Mahmoud, “FPGA-based lane detection system incorporating temperature and light control units,” arXiv:2510.24778, 2025.

[13] W. Liao, “Computational physics framework for real-time imaging denoising: FPGA-based DRAM optimization with high-level synthesis,” *Computer Physics Communications*, vol. 328, Art. no. 110312, 2026, doi: 10.1016/j.cpc.2026.110312. Preprint: arXiv:2508.14917.

[14] T. Sharanya and M. Sampath, “FPGA implementation of edge detection algorithms for image processing,” *International Journal of Innovative Science and Research Technology*, vol. 10, no. 12, pp. 2281–2287, published Jan. 3, 2026, doi: 10.38124/ijisrt/25dec1562.